



TECHNISCHE HOCHSCHULE NÜRNBERG
GEORG SIMON OHM

Datenbanksysteme

Lecture 04

Prof. Dr. Thomas Schedel



Nutzungshinweis

Diese Unterlagen sind **ausschließlich** zu Zwecken der Lehre bestimmt.

Jegliche Vervielfältigung und Verbreitung zu gewerblichen oder anderen Zwecken oder sonst gegen Entgelt sowie die elektronische Zugänglichmachung außerhalb des Intranets der Technischen Hochschule Nürnberg sind **unzulässig**.

Inhalt

- Theorie
 - T2. Das relationale Modell
 - T2.2 Die relationale Algebra
 - T2.2.8 Implementierung

Inhalt

- Praxis
 - P3. Ergebnisse anpassen mittels Single Row Functions
 - P3.1 Single Row Functions
 - P3.2 Character Functions
 - P3.2.1 Case Conversion Functions
 - P3.2.2 Character Manipulation Functions
 - P3.3 Number Functions
 - P3.4 Date Functions

<Theorie>

2.2.8 Implementierung

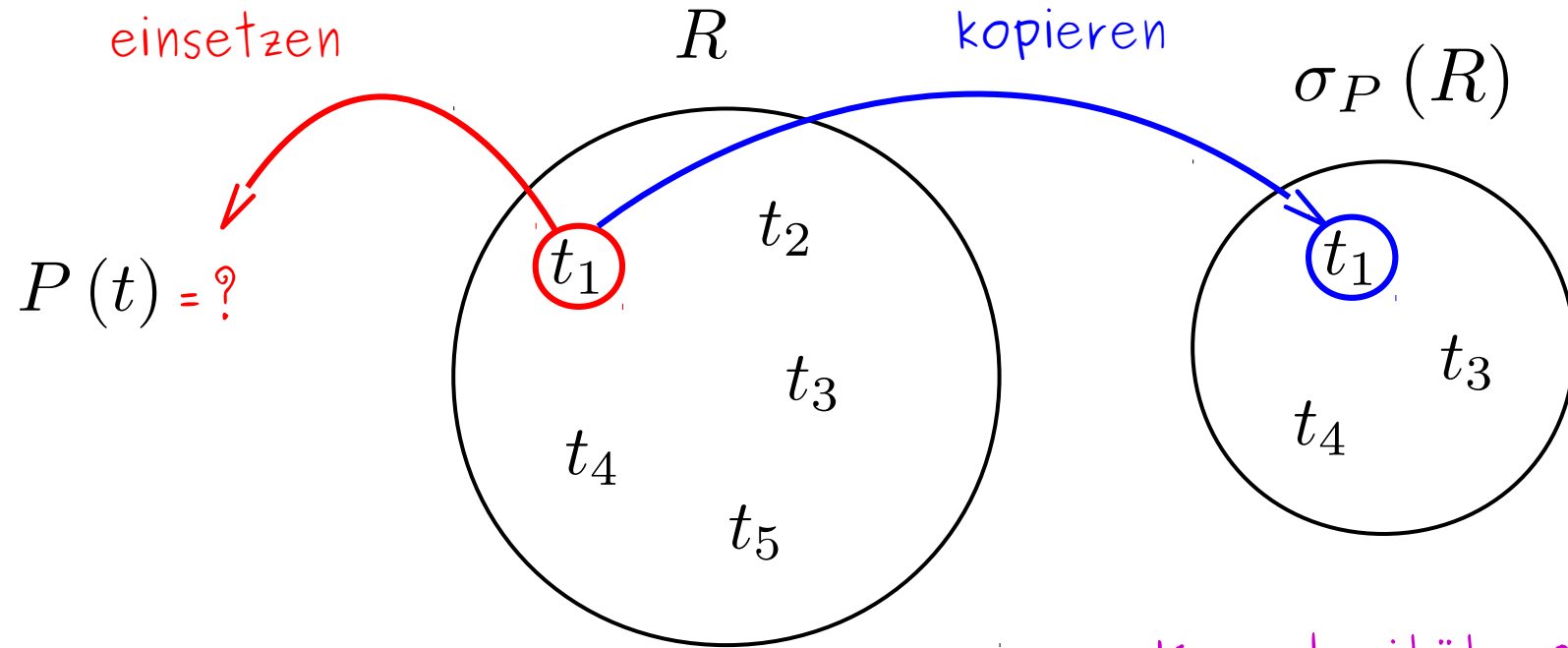
R			
A ₁	A ₂	...	A _n
(a ₁ , a ₂ , ... , a _n)			
.			
.			
.			

$$\sigma_P (R)$$

- x alle Tupel in R durchlaufen
- x für jedes Tupel überprüfen, ob P erfüllt ist
 - ja: zu Ergebnismenge hinzufügen
 - nein: überspringen
- x wenn kein Tupel mehr übrig: Ergebnismenge zurückliefern

1. Attr. Werte
einsetzen

2. $(P(t_1) = \text{wahr})$
kopieren



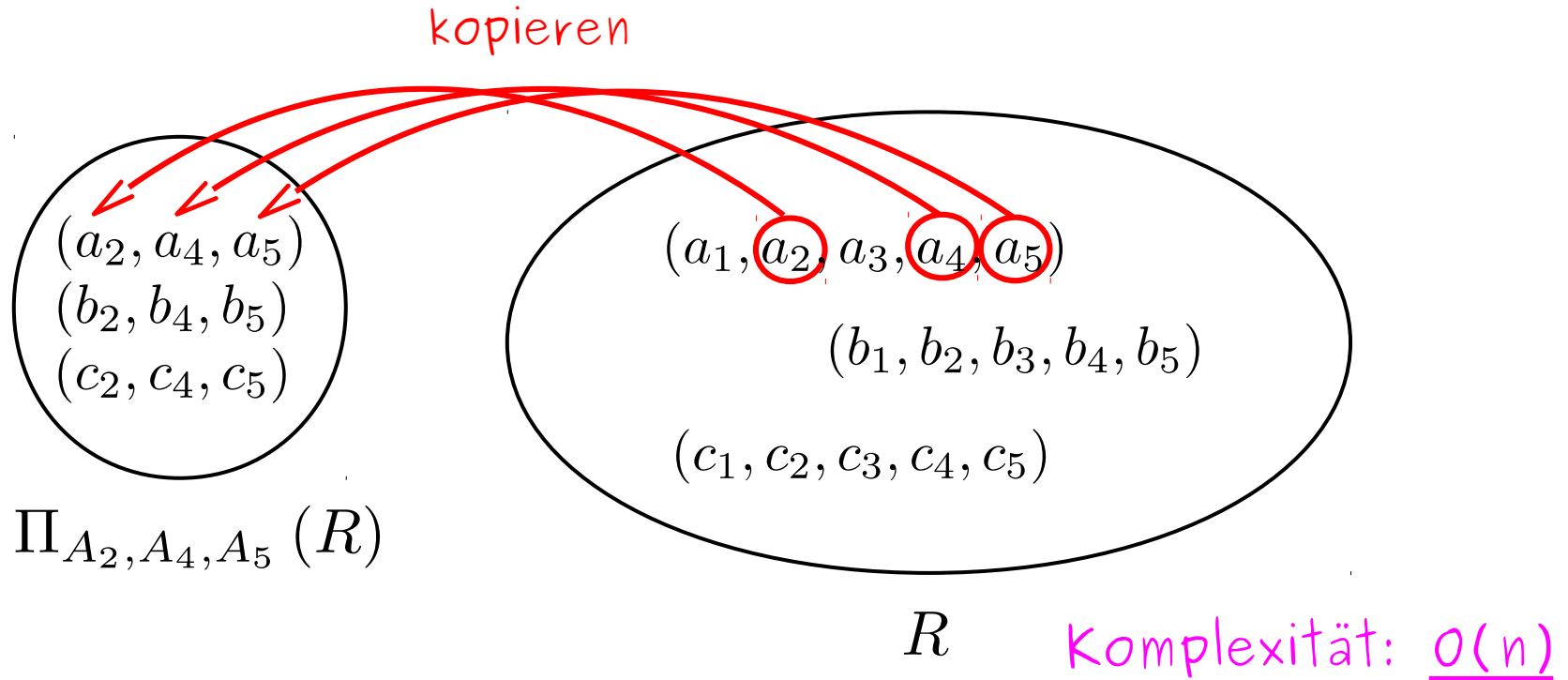
Komplexität: $O(n)$

R			
A ₁	A ₂	...	A _n
(a ₁ , a ₂ , ... , a _n)			
. . .			

$$\Pi_{B_1, \dots, B_m} (R) ; \{B_1, \dots, B_m\} \subseteq \{A_1, \dots, A_n\}$$

- x alle Tupel in R durchlaufen
- x aus jedem Tupel Attributwerte für $B_1, \dots, B_m$ extrahieren
 - neues Tupel erzeugen + zu Ergebnis hinzufügen
- x wenn kein Tupel mehr übrig: Ergebnis zurückliefern

$$\text{sch}(R) = \{A_1, A_2, A_3, A_4, A_5\}$$

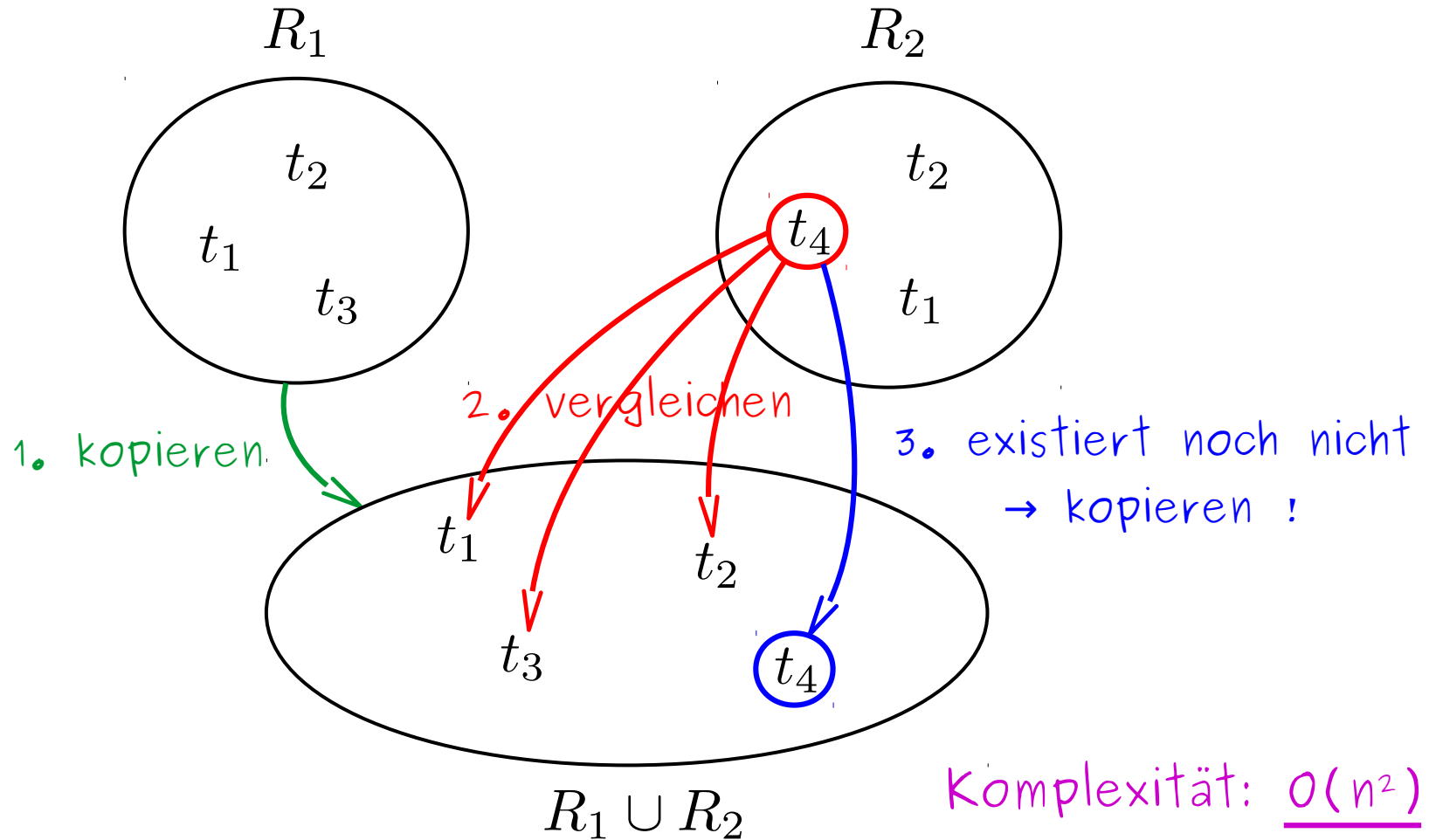


R_1			
A_1	A_2	...	A_n
$(a_1, a_2, \dots, a_n)$			
...			

R_2			
A_1	A_2	...	A_n
$(b_1, b_2, \dots, b_n)$			
...			

$$R_1 \cup R_2 ; \text{sch}(R_1) = \text{sch}(R_2)$$

- x alle Tupel aus R_1 in Ergebnismenge kopieren
- x für jedes Tupel in R_2 überprüfen, ob es bereits in der Ergebnismenge existiert
 - ja: überspringen
 - nein: zur Ergebnismenge hinzufügen
- x wenn kein Tupel in R_2 mehr übrig: Ergebnis zurückliefern



R_1			
A_1	A_2	...	A_n
$(a_1, a_2, \dots, a_n)$			
...			

R_2			
A_1	A_2	...	A_n
$(b_1, b_2, \dots, b_n)$			
...			

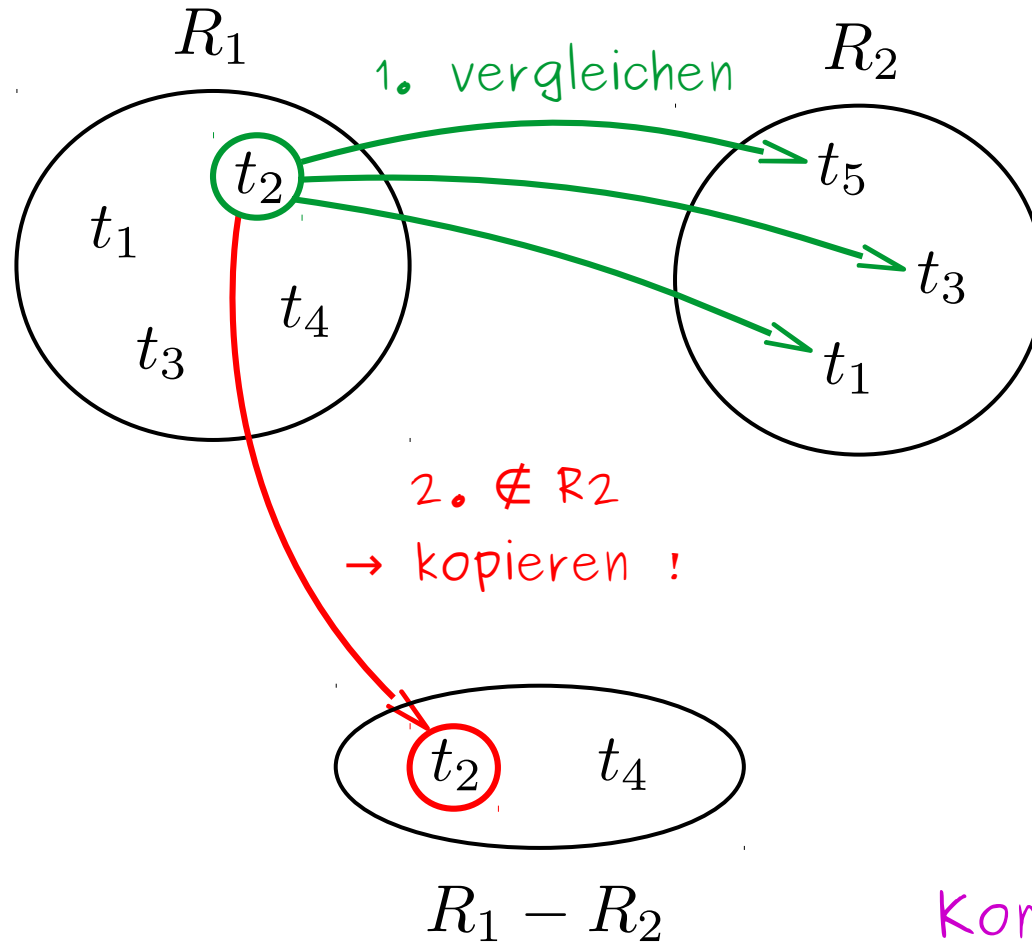
$$R_1 - R_2 ; \text{sch}(R_1) = \text{sch}(R_2)$$

x für alle Tupel aus R_1 überprüfen, ob sie bereits in R_2 vorkommen

→ ja: überspringen

→ nein: zur Ergebnismenge hinzufügen

x wenn kein Tupel in R_1 mehr übrig: Ergebnis zurückliefern



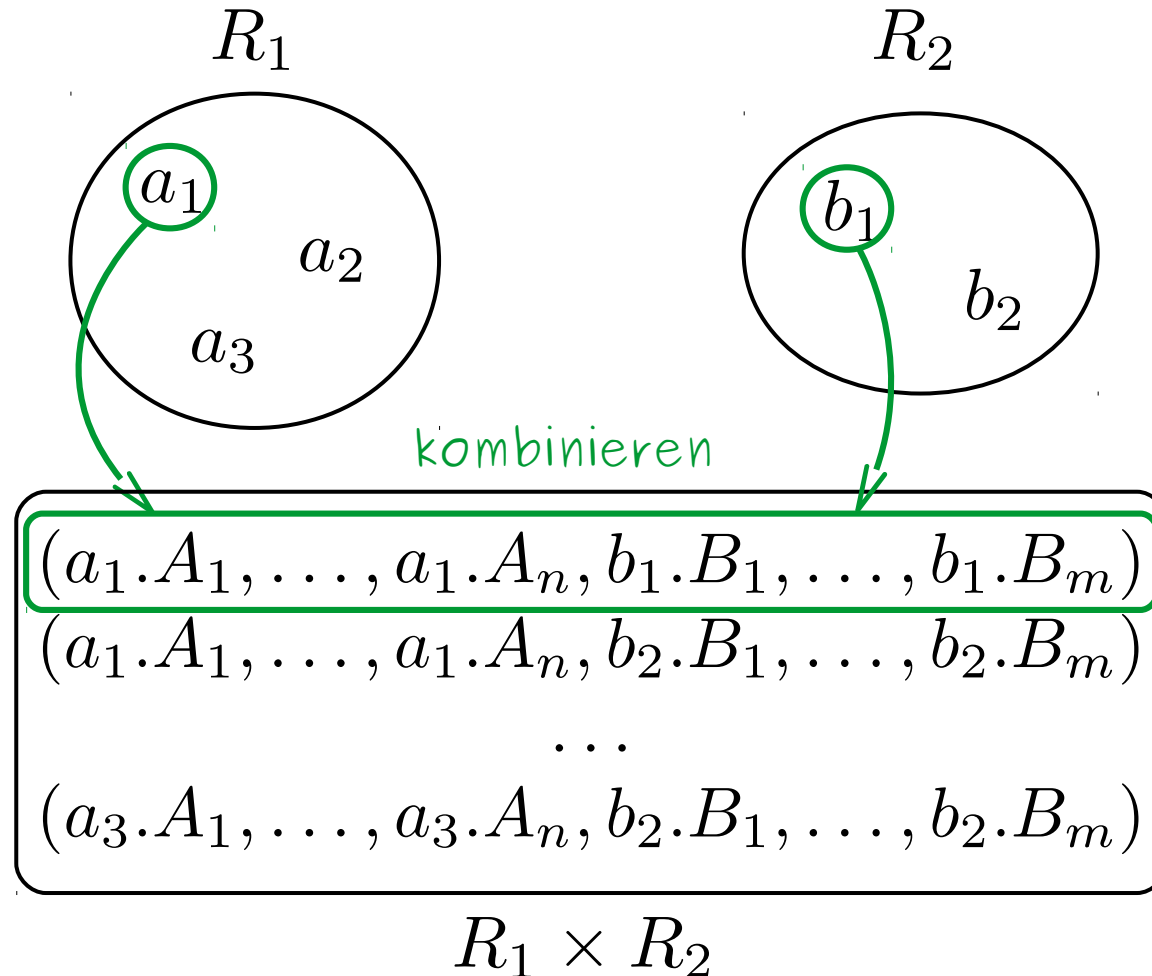
Komplexität: $O(n^2)$

R_1			
A_1	A_2	...	A_n
$(a_1, a_2, \dots, a_n)$			
...			

R_2			
B_1	B_2	...	B_m
$(b_1, b_2, \dots, b_m)$			
...			

$$R_1 \times R_2$$

- x für jedes Tupel $a \in R_1$ alle Tupel $b \in R_2$ durchlaufen
- x a u. b zu neuem Tupel kombinieren
- x neues Tupel zu Ergebnismenge hinzufügen
- x wenn kein Tupel aus R_1 mehr übrig: Ergebnis zurückliefern



Komplexität:
 $O(n^2)$

Theorie

$$R = \left\{ \begin{array}{l} (a_1, \dots, a_n), \\ (b_1, \dots, b_n), \\ (c_1, \dots, c_n) \end{array} \right\}$$

"echte" Menge



Tupel auswählen,
die P erfüllen



fertig!

Praxis

T			
A ₁	A ₂	...	A _n
a ₁	a ₂	...	a _n
b ₁	b ₂	...	b _n
c ₁	c ₂	...	c _n

Tabelle



Tabellenzeilen/Datensätze
auswählen, die P erfüllen



Duplikate eliminieren

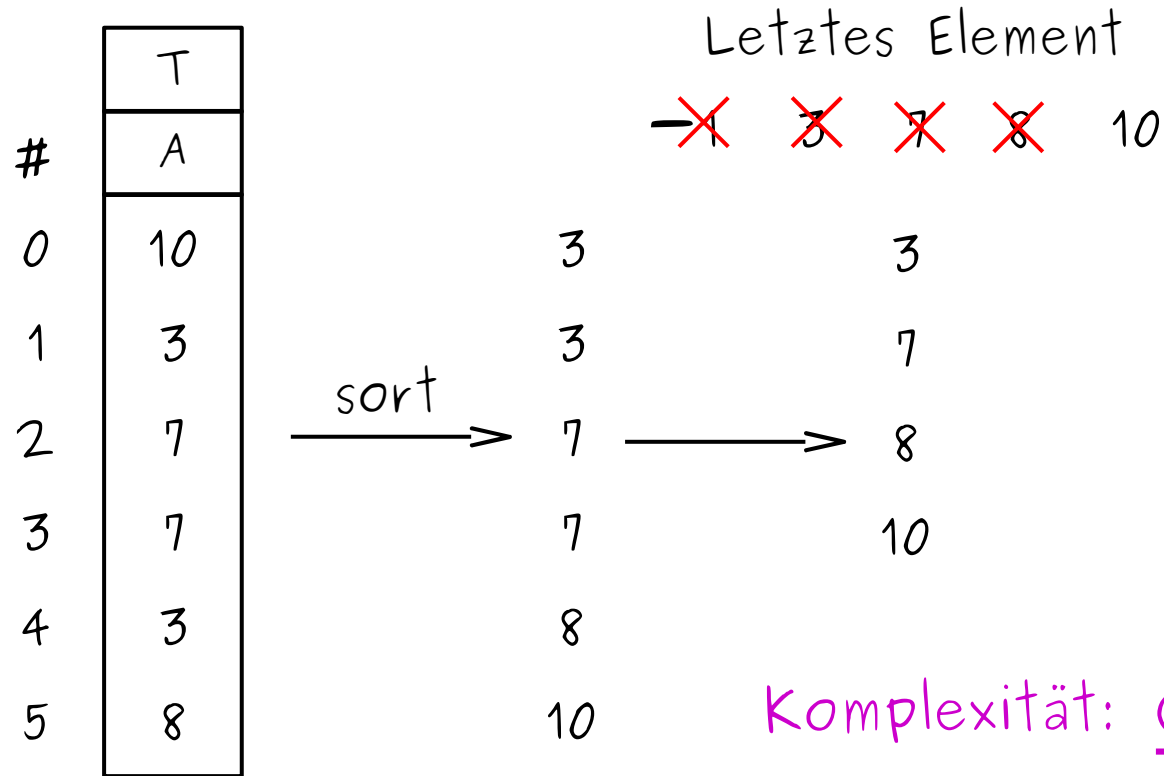


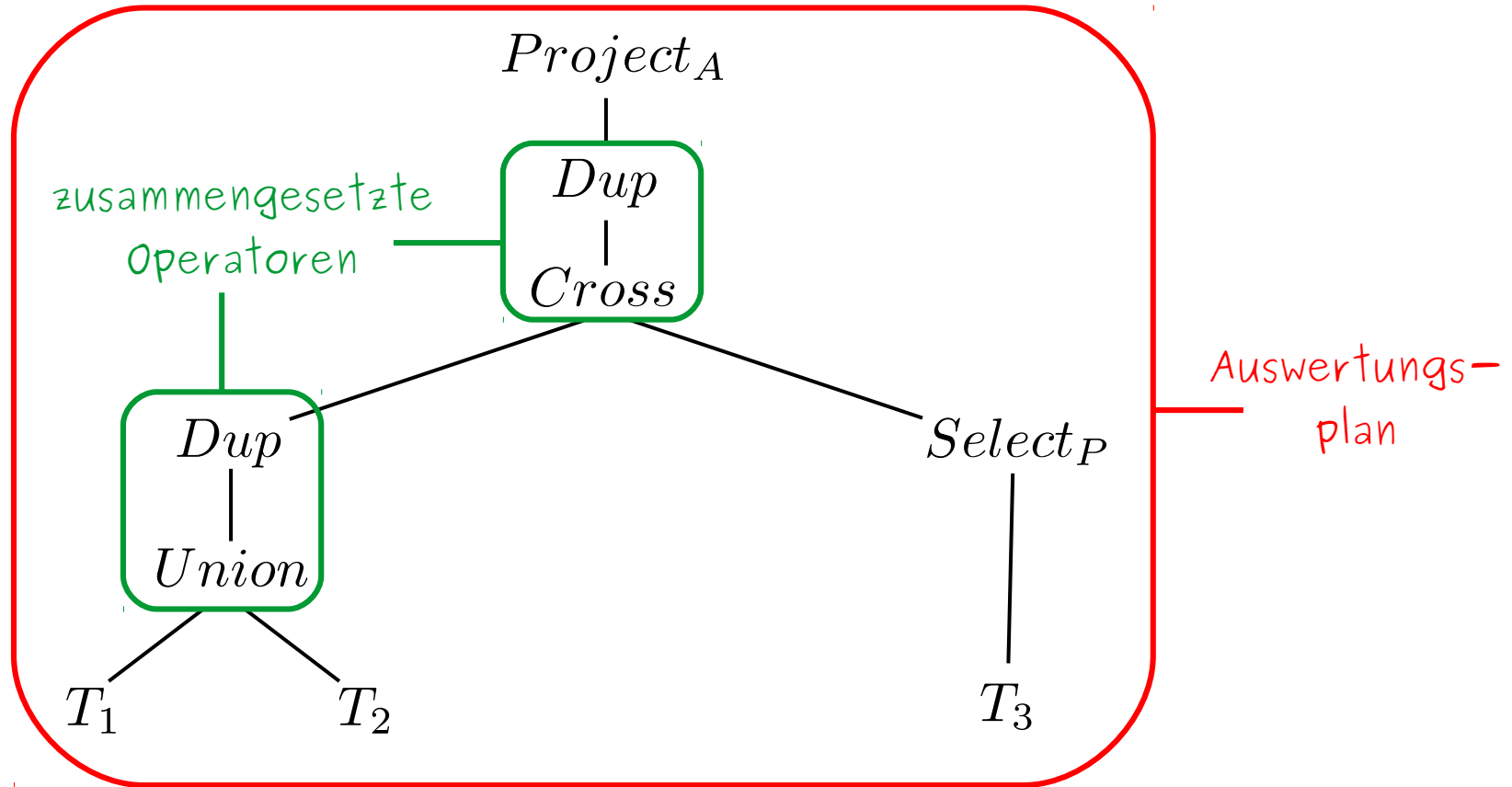
fertig !

Kritisch!

	T	
#	A	
0	10	10
1	3	7
2	7	3
3	7	8
4	3	
5	8	

Komplexität: $O(n^2)$

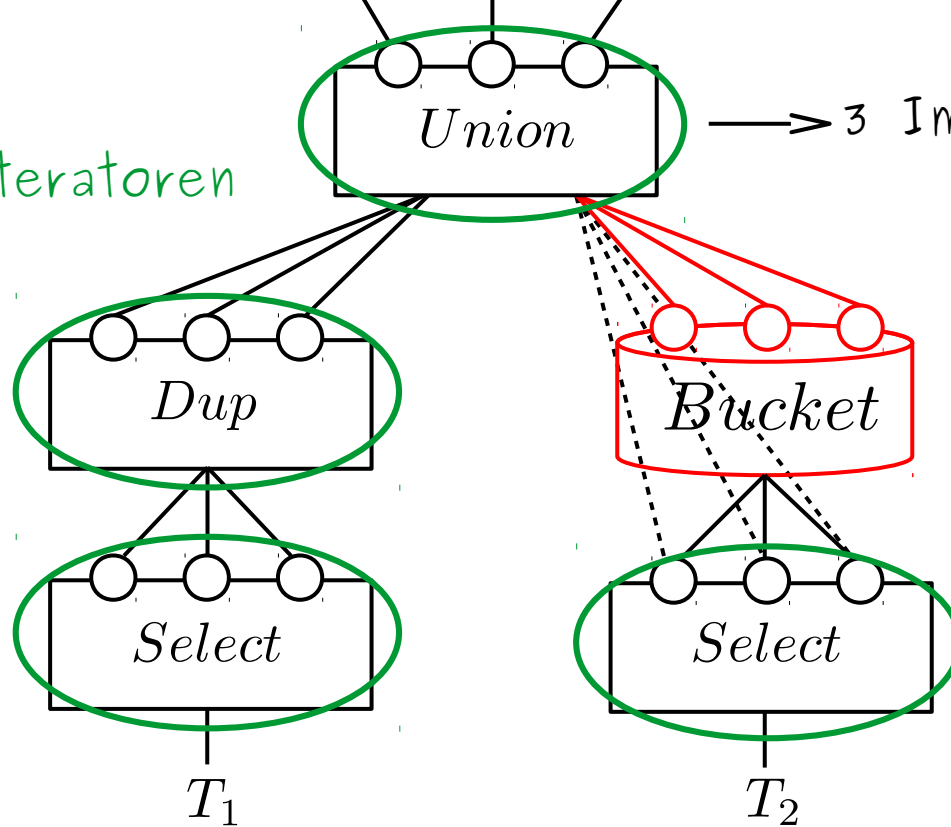


$$Project_A (Dup (Cross (Dup (Union (T_1, T_2)) , Select_P (T_3))))$$


3 Interface-Methoden
open next close

"Brute Force"-Variante

* Iteratoren



→ 3 Impl.: Union, sortUnion, IndexUnion

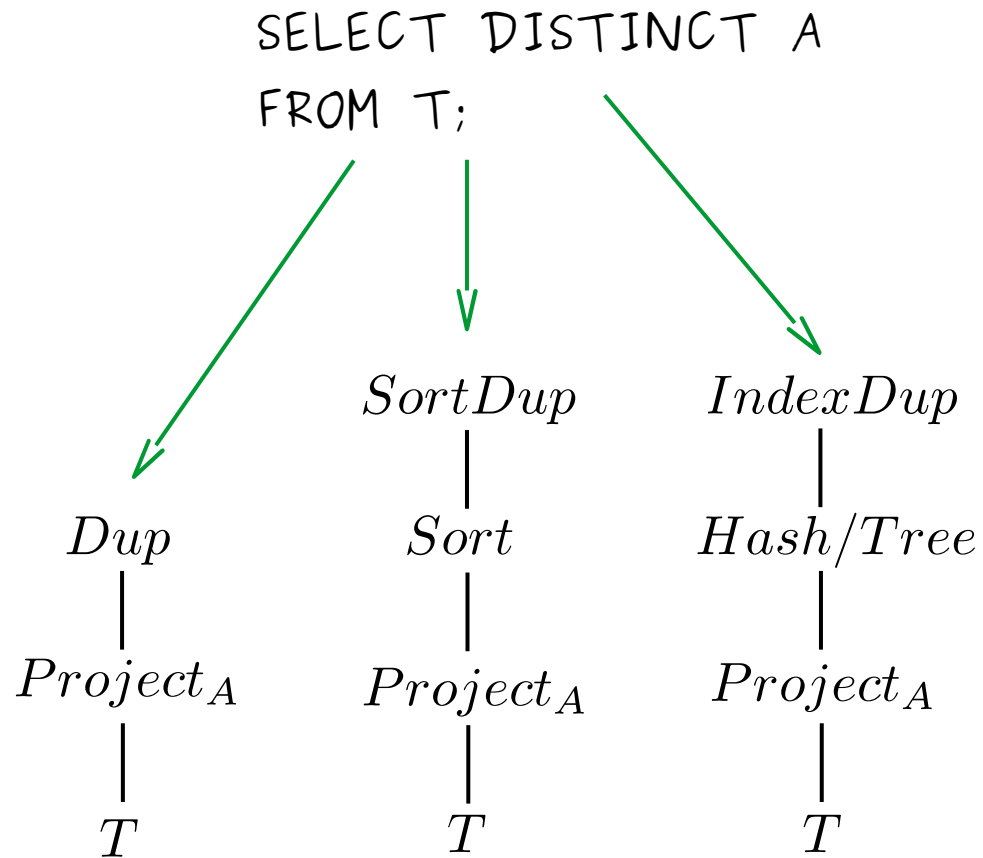


unterschiedl. Buckets
erfordern auch unterschiedl.
Iteratoren-Implementierungen

spezielle Buckets:

- * Sort
- * Hash
- * Tree (z.B. B-Tree)

(Auswertungsstrategie: "Pipelining")



</Theorie>

<Praxis>

P3. Ergebnisse anpassen mittels Single Row Functions

2 Typen von SQL_Funktionen

1) Single Row Functions

- x liefern 1 Ergebnis pro Zeile

2) Multiple Row Functions

- x liefern 1 Ergebnis für Menge von Zeilen

P3.1 Single Row Functions

- x manipulieren Daten
- x arbeiten auf jeder Zeile des (Zwischen-)Ergebnisses
- x liefern 1 Ergebnis pro Zeile
- x können Datentyp verändern
- x akzeptieren Spalten u. Ausdrücke als Argumente

5 Kategorien: Character, Number, Date, Conversion, General

P3.2 Character Functions

Sehen wir uns heute an

2 Kategorien:

1) Case Conversion Functions

x LOWER, UPPER, INITCAP

2) Character Manipulation Functions

x CONCAT, SUBSTR, LENGTH, INSTR,
LPAD, RPAD, TRIM, REPLACE

P3.2.1 Case Conversion Functions

Funktion	Ergebnis
LOWER('SQL ist cool')	'sql ist cool'
UPPER('SQL ist cool')	'SQL IST COOL'
INITCAP('SQL ist cool')	'Sql Ist Cool'

z.B.: SELECT MitarbeiterNr, Vorname, Nachname
FROM Mitarbeiter
WHERE LOWER(Nachname) = 'müller';

P3.2.2 Character Manipulation Functions

Funktion	Ergebnis
CONCAT('Hello', 'World')	'HelloWorld'
SUBSTR('HelloWorld', 1, 5)	'Hello'
LENGTH('HelloWorld')	10
INSTR('HelloWorld', 'W')	6
LPAD(Gehalt, 10, '*')	'*****2400'
RPAD(Gehalt, 10, '*')	'2400*****'

falls string nicht
vorkommt: 0

Funktion	Ergebnis
REPLACE('SQL ist doof' , 'doof' , 'cool')	'SQL ist cool'
TRIM('H' FROM 'HelloWorld')	'elloWorld'

z.B.: SELECT MitarbeiterNr, CONCAT(Vorname, Nachname) Name
 , LENGTH(Nachname)
 , INSTR(Nachname, 'a') "erstes 'a'"
FROM Mitarbeiter
WHERE SUBSTR(Funktion,1,6) = 'Senior';

P3.3 Number Functions

Funktion	Ergebnis
ROUND(37.176,2)	37.18
TRUNC(37.176,2)	37.17
MOD(1100,300)	200

Tipp: Test mit Tabelle DUAL

z.B.: `SELECT ROUND(37.176,2)`
`FROM DUAL;`

P3.4 Date Functions

z.B.: SYSDATE

Default-Uhrzeit
ist immer 0:00 Uhr

MONTHS_BETWEEN(SYSDATE, '01-10-2222')

ADD_MONTHS('01-10-2222', 1)

NEXT_DAY('01-10-2222', 'Dienstag')

LAST_DAY('01-10-2222')

Einheit: Tage

Auch möglich: $(\text{SYSDATE} - \text{Einstellungsdatum}) / 7$

↳ Tagesgenaues Rechnen !

Tipp: auch hier Test mit Tabelle DUAL

z.B.: SELECT SYSDATE FROM DUAL;

</Praxis>



Datenbanksysteme

Lecture 04

`exit(0);`

